Chapter 4

Scalable Data Pipelines and Feature Management

4.1 The Bug That No Alert Can Catch

Google Play once had a production problem that was hidden from view: the recommendation models were running, requests completed, predictions arrived, and the dashboards showed no obvious service failure. However, a few feature fields that were populated in the training data had no corresponding values in the serving logs, which meant that the model had learned under one set of conditions and was making predictions under another.

Google found the discrepancy by comparing feature statistics from training data with the serving-log statistics for the same day. Once the team removed the skew and gave the model consistent inputs, an online A/B experiment on the app store's main landing page measured a 2% increase in app installs (Baylor et al., 2017).

Two percent on a surface as large as Google Play is a major product result, although the more instructive detail for a recsys engineer is how easily the defect stayed hidden. An availability monitor could confirm that the prediction service returned a response, a latency monitor could confirm that the response arrived quickly, and an exception tracker could show that no code had crashed, while the model quietly operated on defaults for values it never received. The issue was eventually found by a data-validation component that compared training and serving statistics for the same day; a team that monitored each dataset separately, without comparing the two contexts, could easily have missed it.

Failures like this occur because a production machine learning system is much larger than its model. A recommender may represent a user through recent clicks, long-term category preferences, subscription status, location, device context, and hundreds of item attributes, with each input originating somewhere outside the model. Raw events have to be collected, cleaned, aggregated, stored, transported, and reconstructed at prediction time before the model can use them. A feature is a value derived from that data and presented to a model, such as the number of purchases a user made during the last 30 days, and the code and infrastructure that produce those values form a second system around the trained model.

That surrounding system often determines whether the model succeeds. After building its Michelangelo machine learning platform, Uber summarized the problem thus: finding useful features was often the hardest part of machine learning, while building and managing data pipelines was among the most expensive parts of a complete solution (Del Balso and Hermann, 2017). This observation came from operating models for products such as estimated arrival times and meal delivery, where an apparently ordinary feature like a restaurant's average preparation time requires historical computation, reliable updates, and low-latency access.

The industry did not begin with a shared architecture for this work, so teams at Google, Netflix, Uber, Airbnb, LinkedIn, Pinterest, GoJek, Etsy, and DoorDash encountered the same class of problems from different directions. Their systems had different names and implementation choices, yet their incident reports describe four recurring failures that are useful to follow in the order a team usually experiences them.

The first failure usually appears when features built in offline analytical systems, such as BigQuery, Snowflake, or a data lake, need to become available inside a user-facing request that has only a few milliseconds to spare. Solving that latency gap creates separate training and serving paths, which can compute different values for a feature carrying the same name. The training pipeline can also use information that was unavailable when a historical prediction would have been made, and even when a team understands both risks, the safest methods for maintaining consistency may force it to wait months for data or maintain duplicate implementations.

These failures sometimes reinforce one another. Solving the latency problem creates separate storage and computation paths, those paths introduce opportunities for inconsistency, reconstructing historical values adds temporal correctness requirements, and meeting all of those requirements with hand-built pipelines slows every experiment. The resulting industry pattern resembles convergent evolution in biology: organizations facing similar environmental pressures independently developed systems with similar capabilities, even when their implementations and terminology differed.

We will first walk you through the four failure modes that created this convergence, beginning with the most obvious one: the gap between the offline SQL query against the analytical warehouse used to create the training dataset and a realtime serving request. We will then assemble the resulting capabilities into the broader feature-store pattern, examine when a smaller solution is sufficient, and cover the freshness, monitoring, ownership, and governance practices required once features become shared production dependencies. The goal is to help an ML engineer reason about both the technical boundary and the cross-functional work with data engineering, platform, and application teams, rather than treating a feature store as a product that should be adopted by default.

4.2 Failure 1: You Cannot Run SQL at Request Time

Consider what happens after a user submits a search query on Etsy. The system may retrieve about 1,000 candidate listings, obtain roughly 300 features for every candidate, send those values to machine learning services for scoring, rank the listings, and return the result. These are figures reported directly by Etsy for its search-ranking workload, along with an end-to-end budget of 250 milliseconds and a goal of completing most requests within 75 milliseconds (Walecka and Gallatin, 2023).

One request can therefore require hundreds of thousands of feature values before ranking even begins, while the same latency budget must cover candidate retrieval, network communication, model inference, business rules, and response construction. A feature lookup that takes seconds is clearly unusable, and even individually fast calls can exhaust the budget when the service repeats them for every candidate.

Analytical warehouses are designed for a different workload. A warehouse such as BigQuery or Snowflake stores large historical datasets and efficiently performs scans, filters, aggregations, and joins, making it well suited to questions like, "What was each user's average order value during the previous 30 days?" That query may process billions of rows and produce millions of results, whereas interactive recommendation serving asks, "For these particular users and items, return the values that have already been computed." Bulk processing rewards efficient work across a large dataset; serving requires predictable point lookups under high concurrency.

Monzo, a digital bank based in the United Kingdom, initially crossed this boundary in the most direct way. Some aggregates, including a customer's seven-day average balance, existed in BigQuery but were unavailable to production services, so its machine learning team queried BigQuery from those services and cached the results. This worked for the first few features, but as the same requirement appeared

across projects, the team realized that it was repeatedly building a bridge from analytical storage into production storage (Lathia, 2020).

Caching can soften repeated access, although it does not change the warehouse's underlying workload or reliability characteristics. A cache miss still invokes a query whose latency may vary with data volume, query planning, and shared capacity, while invalidation raises a separate set of freshness questions. Different caches can serve values from different update times, and a burst of misses can transfer load back to the warehouse precisely when the online service is already under stress. You have also coupled a user-facing request path to a system whose availability objectives were never designed around that request.

Uber encountered the same boundary with operational data. Its engineers explained that an online model could not query the Uber Eats order service and calculate a restaurant's average meal preparation time over a historical window, since doing so would scan and aggregate past orders while adding load to a service built to manage current ones. The model still needed the feature, so Uber computed it before the prediction request and stored the result where the serving system could retrieve quickly (Del Balso and Hermann, 2017).

This technique is called materialization: a data pipeline computes a feature and writes its value into storage optimized for later reads. For an entity such as a user, restaurant, or item, the serving store usually keeps the latest available value under a key derived from the entity identifier. A key-value store can retrieve that value directly, without performing a scan or join during the request, which is why Redis, Cassandra, Bigtable, and similar systems are common choices for high-throughput lookups with predictable latency.

Materialization changes the request path from computation into retrieval. Rather than scanning order history to calculate `restaurant_avg_prep_time_7d`, the service fetches the latest precomputed value for a restaurant; rather than joining 1,000 candidate identifiers with several warehouse tables, it submits those identifiers in a batched request. Batching combines many lookups into one network round trip and gives the storage system an efficient unit of work.

After candidate generation, the ranking service usually assembles its model input from three groups. The request supplies context such as the user identifier, query, device, session, and location, while the candidate-generation service supplies a list of item IDs selected for ranking. These item IDs are called candidate item IDs in this stage. The ranking service uses the user ID and candidate item IDs to perform batched lookups of precomputed user and item features from an online store such as Redis or Cassandra. A feature service may return these values as a JSON object, a Protocol Buffers message, or another serialized structure that the serving code validates and converts. The service may also compute on-demand features that combine the request with each candidate, such as query-item similarity or geographic distance. It then assembles the request context, retrieved features, and on-demand features into the tensor or feature vector expected by the model.

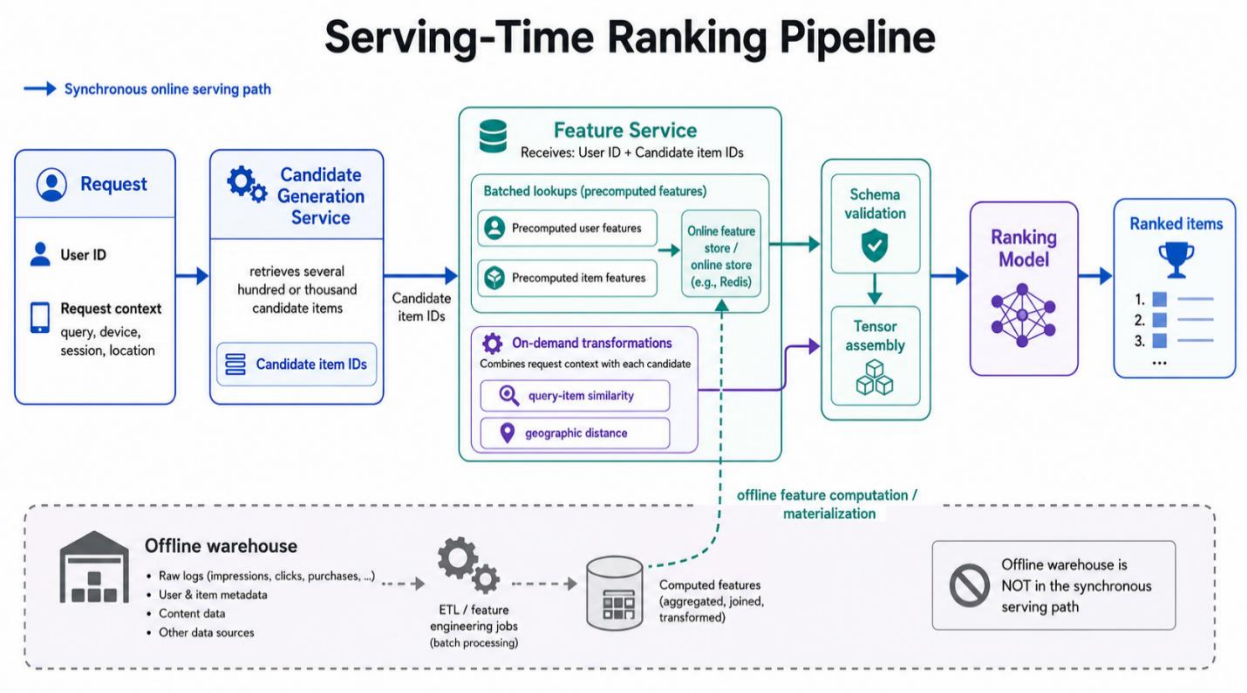


Figure 4.1: A realtime model commonly combines request-scoped context, precomputed values from an online store, and on-demand transformations before assembling the model input. The analytical warehouse remains outside the synchronous request path.

DoorDash's reported scale shows why this pattern becomes necessary. Its store-ranking use case made more than one million predictions per second and used dozens of features per prediction, resulting in tens of millions of feature reads per second, while a characteristic prediction request involved about 1,000 feature lookups. The complete system contained billions of feature-value pairs (DoorDash, 2020). At that scale, data layout affects latency and infrastructure cost, so DoorDash retained Redis after benchmarking several databases and changed its representation to use Redis hashes with compact feature identifiers. The redesign reduced read latency by more than 40% and sharply reduced memory and CPU consumption (DoorDash, 2020).

The resulting architecture assigns two different roles to storage. An offline store retains historical feature data for training, analysis, and backfills, usually in a warehouse, lake, or lakehouse where large scans and joins are acceptable. In contrast, an online store arranges values for prediction-time access and usually keeps the latest value for each entity and feature. Both stores may contain logically equivalent features, but their physical layouts differ because the access patterns they support are different.

Feature computation also tends to have two update paths. Batch pipelines process a bounded collection of records on an hourly or daily schedule and are suitable for features such as a user's 90-day purchase count or an item's long-term popularity. Streaming pipelines process events continuously as they arrive, which makes them appropriate for clicks during the last ten minutes or current session activity. Both can materialize values into the same online store, while request-scoped features such as the current query text, device type, or cart contents are normally computed from the request because they do not exist beforehand.

This division is sometimes called a lambda architecture, where batch computation supplies comprehensive historical processing and streaming computation supplies recent updates. The name is

less important than the constraint that created the design: historical data is efficient to process in bulk, recent events may need to influence a recommendation quickly, and the serving path can afford neither a warehouse query nor a large aggregation.

The choice between batch, streaming, and request-time computation can be made per feature rather than once for the whole platform. Expensive values reused across many requests should generally be computed ahead of time; when they can tolerate hours of delay, a batch pipeline is sufficient, while fresher values may require streaming materialization. Cheap computations that depend on the current request can stay on the read path, and low-cost, slow-changing features can use whichever existing path is simplest to operate.

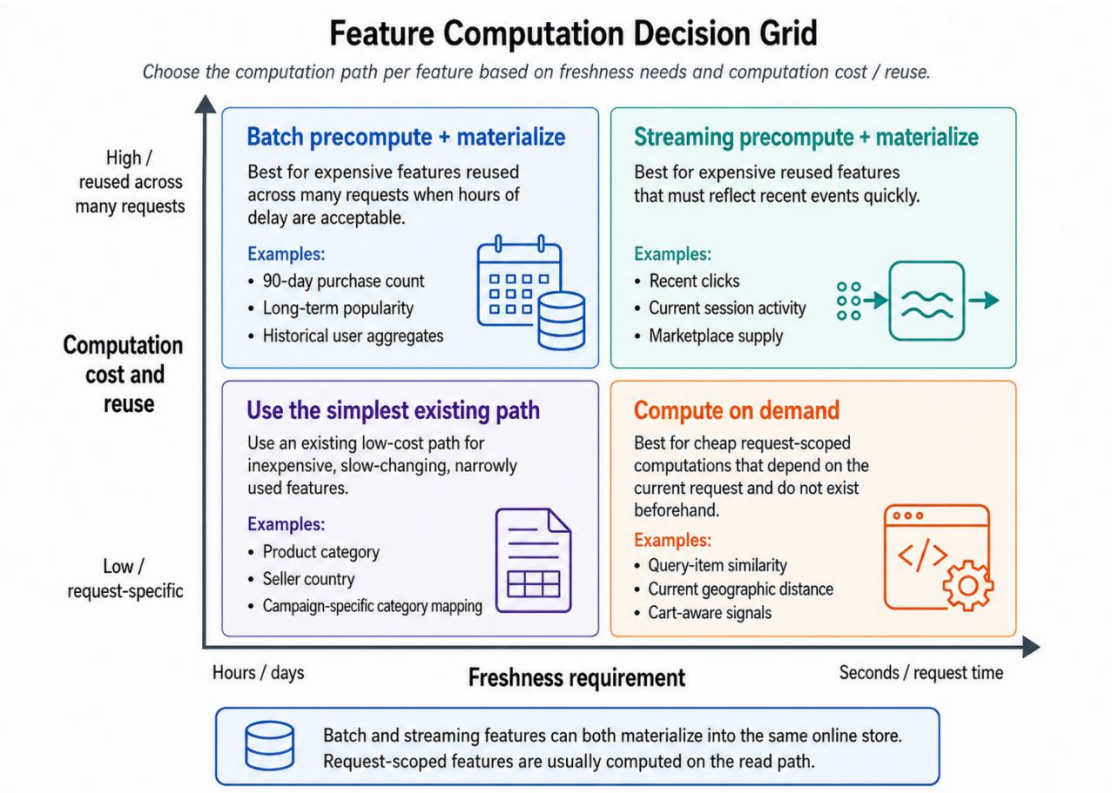


Figure 4.2: A per-feature decision grid for choosing batch materialization, streaming materialization, or request-time computation. Freshness, computation cost, and reuse determine the appropriate path.

A robust request path therefore avoids analytical joins. It collects the relevant entity keys, issues one or a small number of batched reads, combines the returned values with request-scoped context, and invokes the model, leaving expensive transformations to managed pipelines that ran earlier. The online service now performs bounded work whose latency the team can measure and control.

Although this design addresses the immediate latency problem, it also creates a deeper consistency problem. Training reads historical values from the offline world and serving reads current values from the online world, so different computation or transport code can give one feature two meanings while preserving its name. Google Play's missing features were one instance of this divergence.

4.3 Failure 2: It Worked Offline, but Failed Online

The Google Play incident from the chapter opening is the clearest example of this second failure: some feature values were present in training and missing when the model was served. Missing values are only one form of training-serving skew, however, since a feature can also exist in both environments while being computed from different data, code, defaults, or time windows.

GoJek (a popular ride-hailing platform in Asia) has a driver-allocation system, Jaeger, which matched millions of customers with hundreds of thousands of drivers each day. Data scientists developed feature transformations in Python and stored the outputs in the data warehouse, while production services needed separate implementations. Engineers rewrote the transformations for online serving, provisioned databases and ingestion jobs, and repeated the work across models, projects, environments, and regions. GoJek later reported that these duplicated transformations created inconsistencies that led to training-serving skew (Lim, 2020).

Training-serving skew means that a model systematically receives different inputs during training and serving. The discrepancy can come from transformation logic, source data, timing, default values, or feedback introduced by the deployed system. A useful test is to take the same entity at the same logical time and ask whether both pipelines would produce the same feature value.

The answer can be no even when each implementation seems reasonable on its own. An offline SQL query might define a seven-day count with an inclusive lower boundary, while a streaming job excludes events exactly seven days old; a Python pipeline might fill a missing category with `"unknown"`, while a Java service uses an empty string. Similar errors occur when one path measures currency in dollars and another in cents, or when clipping and logarithmic transformations run in a different order. Each difference shifts the input distribution seen by the model.

Etsy encountered a subtle version of this problem in timestamp representation. Timestamps served with millisecond precision were interpreted by Pandas, NumPy, and Spark as datetime objects with nanosecond precision, so a field that still appeared to be a timestamp carried a risk of framework-specific disagreement between training and inference. Etsy standardized the representation as Unix time in milliseconds stored in a plain integer type (Gallatin and Starr, 2023). Very little code is required to create skew; a difference in precision can be enough.

Diagnosis is difficult because the model will usually accept these values. Missing numeric features may become zero, categorical values may map to an out-of-vocabulary bucket, and a changed scale still produces a valid floating-point number, which leaves the prediction service healthy at the software level. Model quality declines while the labels needed to reveal that decline may arrive hours or days later. Airbnb's engineers described the consequence directly: a model consumes inconsistent data and produces inaccurate predictions, while the team can remain unaware until labels become available (Simha, 2020).

The Google Play incident from Section 4.1 can now be interpreted through a mechanical explanation. Features existed during training but were always absent from serving logs, and the model or preprocessing code could have replaced those missing values with defaults before returning scores as usual. Comparing training and serving statistics for the same day exposed the mismatch, and removing it increased app installs by 2% (Baylor et al., 2017).

Production teams generally combine three defenses because each one catches a different part of the problem. Start by logging the features actually used for a prediction, including the entity, prediction time, model version, and the feature vector presented to the model. Once the outcome is available (e.g., clicks or conversions), those logged features can be joined with the labels and reused for training.

Google recommends this approach in its Rules of Machine Learning, and YouTube's home page reported significant quality improvements together with lower code complexity after adopting serving-time feature logging (Zinkevich, 2017).

Training from serving logs gives you a strong consistency property because the model learns from the same representation that production supplied. It also preserves values from external services and request-time context that may be difficult to reconstruct later, but the trade-off is waiting. A new feature has no historical serving logs, so the team must deploy the model first (accepting the risk of a train-serve skew), collect values, and postpone training until enough labeled examples accumulate. The logging system also needs careful sampling, privacy controls, versioning, and reliable label joins.

Shared transformation code provides another layer of protection by using one definition in both training and serving contexts. Netflix[1] followed this pattern in DeLorean, reconstructing historical service inputs and running the same Java feature encoders used online, and its later Axion fact store continued sharing feature encoders between online computation and offline generation (Netflix Technology Blog, 2016; Kaushal, 2022). Pinterest built its Linchpin domain-specific language for the same reason, allowing one feature transformation to run in both environments and removing a class of mismatches caused by parallel implementations (Pinterest Engineering, 2025).

Shared code still requires a shared execution contract. The same function can produce different values when it receives different source records, uses another library version, interprets time zones differently, or reads mutable state. Teams therefore standardize schemas, types, defaults, feature versions, and event-time semantics around the code. A schema defines the expected name and type of every field, while validation rejects or flags records that violate those expectations before they reach the model.

Continuous measurement completes the set of defenses. The system samples online feature fetches, reconstructs the values through the offline path for the same entity and timestamp, and compares both results. Airbnb's Chronon performs this replay-and-diff process by using logged online requests as the keys and timestamps for an offline backfill (Simha, 2023). Exact comparisons work for discrete values, tolerance checks handle floating-point values, and distribution-level checks compare missing rates, ranges, means, quantiles, and category frequencies between training and serving.

[1] Netflix is extensively used as the canonical example in this chapter for a couple of reasons: A) they were pioneers in implementing recommender systems at scale nearly 2 decades ago and hence the technical problems they face along the way tend to be quite at the leading edge. B) They have very good engineering blogs that are a gold mine of design choices behind the solutions, rather than taking the solutions at face value.

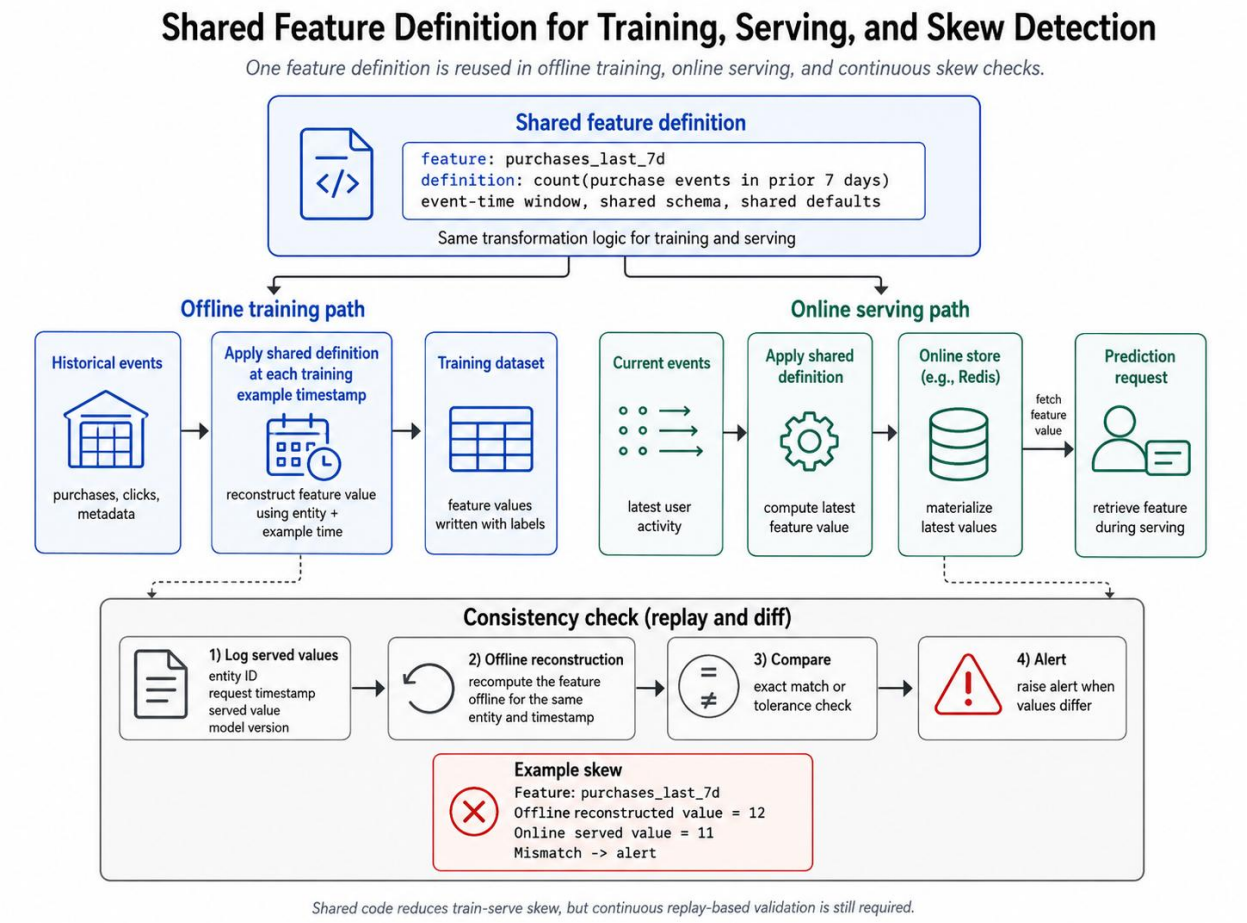


Figure 4.3: Shared definitions reduce training-serving skew, while replay and comparison detect differences caused by data, timing, dependencies, or execution environments.

Measurement remains necessary after code is shared because system and data changes can introduce skew after launch, which is why Google recommends monitoring the gap explicitly (Zinkevich, 2017). An upstream schema change, a stalled streaming consumer, an expired cache entry, or an altered default can change serving values without touching feature code. The invariant you want to observe is equivalent behavior, rather than simply equivalent source code.

These defenses improve consistency between training and serving systems, but they cannot prove that the training data represents the world as it existed when each historical prediction should have occurred. An offline pipeline may reproduce its own logic perfectly while still giving the model information from the future.

4.4 Failure 3: The Model That Saw the Future

In the 2016 Kaggle Expedia Hotel Recommendations competition, one legitimate-looking column created an extraordinary shortcut. `orig_destination_distance` represented the physical distance between the customer and the hotel destination, so distance itself was valid information that could have been known when the user searched. The leak came from the way the competition data had been constructed: when competitors combined this high-cardinality distance with the user's location and

hotel market, they could find exact or near-exact rows in the training data and copy the associated hotel cluster for roughly one third of the test data (Mavalankar et al., 2019).

This was a train-test leakage caused by record linkage, rather than an example of the destination being derived from a future booking. It is still a useful illustration because it shows that a feature can be individually valid while the way datasets are joined or split creates an answer key. Production feature pipelines face a closely related temporal version of the problem when they join a historical prediction to the latest state of a mutable table.

Data leakage occurs when training uses information that would have been unavailable during a real prediction. Target leakage is the most direct form, where a feature includes the label or a consequence of that label, while temporal leakage covers any feature value incorporating events that happened after the historical prediction time.

Suppose a training row represents a recommendation shown to user 42 at 10:00 on May 1, with a label recording whether the user purchased the item during the following day. Let's say the company has a current user-profile table that identifies user 42 as a high-value customer. If that status was assigned on May 2 after the purchase was made, joining today's latest profile into the May 1 training row gives the model future knowledge. If the model is supposed to predict whether a customer will convert the next day, it is likely to assign a higher propensity to convert to a user already labeled as high value.

Mutable tables make this error common because current subscription tier, risk score, item popularity, seller quality, or account state is overwritten as the world changes. Joining the latest row to historical examples produces a clean table with plausible values, and offline metrics may improve because later state often reflects earlier outcomes. Production then disappoints because the later values are unavailable at prediction time. Google's Rules of Machine Learning warns that a table can change between training and serving and that periodic snapshots provide only an approximation of historical state (Zinkevich, 2017).

The governing requirement is point-in-time correctness, which means that every feature in a training row could have been known at that row's prediction timestamp. Each row needs at least an entity key and event timestamp, while feature history needs timestamps stating when each value became valid. The training join then selects the latest feature record whose timestamp is less than or equal to the prediction time.

This operation is called an as-of join, or temporal join, and for every label row it asks, "What was the most recent known value as of this time?" If a user's purchase count changed from 3 at 09:00 to 4 at 11:00, a training example timestamped 10:00 must receive 3. A conventional join against the latest user table returns 4 and leaks the 11:00 purchase.

Windowed features introduce another temporal boundary. A feature such as `purchases_last_30d` has to be computed over the 30 days ending at each example's timestamp, meaning that two examples for the same user on different days require different windows. Computing today's count once and attaching it to every historical row leaks recent events backward through the dataset.

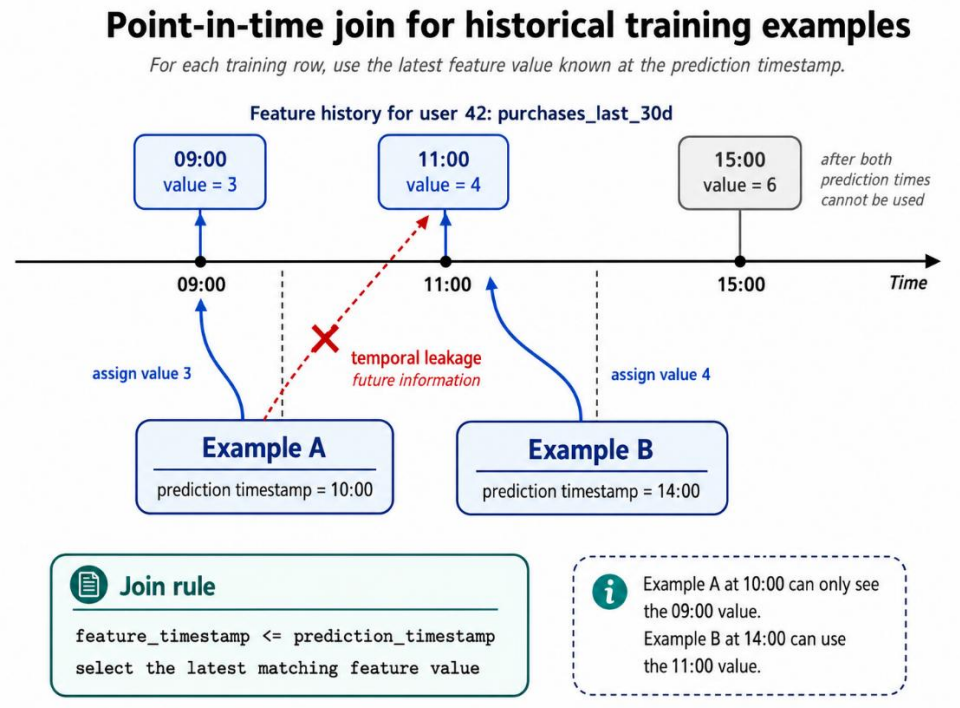


Figure 4.4: A point-in-time join selects the newest feature value available at each historical prediction time and excludes later updates.

Netflix described this requirement memorably while explaining DeLorean: "no paradoxes are allowed," including a label appearing among the features (Netflix Technology Blog, 2016). Its solution captured snapshots of the online services supplying viewing history, saved lists, and predicted ratings. Offline jobs could then reconstruct the inputs seen by the recommender and run the same feature encoders used online. This fact-snapshot approach preserves historical source state and allows features to be regenerated when needed.

Airbnb and LinkedIn followed a second approach by retaining timestamped source data together with declarative feature definitions. A declarative definition states the transformation a team wants, such as a grouped count over a time window, while the platform selects and executes the corresponding batch or streaming plan. Airbnb's Zipline and Chronon and LinkedIn's Feathr could replay these definitions over history and construct point-in-time-correct training data (Simha, 2023; LinkedIn Engineering, 2022).

The reusable asset differs between these approaches. Netflix stores historical facts and reruns shared encoders, while declarative systems store transformation definitions and replay them over timestamped event history. Both enforce the same temporal contract: a historical example can use only information that was available by its prediction time.

Serving logs provide a third option because they capture the exact feature vector used for a prediction and naturally preserve point-in-time values. Their limitation is that no history exists for a feature that has never been served, leaving the team to wait for new logs or reconstruct the feature over historical data. Each choice has traditionally carried a painful cost, which leads to the final failure.

4.5 Failure 4: Six Months of Waiting, or Two Codebases

An Airbnb data scientist summarized feature development before Zipline in one sentence: "I spend 60% of my time generating training data" (Zanoyan and Shapiro, 2018). That number represented much more than the time spent writing SQL because each promising feature idea forced the team to choose between two unsatisfactory paths.

One path was to log the feature and wait. Engineers added it to the online system , recorded its value during real requests (but not use those features in any ML model yet), and waited for labels to arrive. Then join the logged features with the labels to create the training set. The eventual training set had valuable properties: values came directly from the serving path, consistency was strong, and every value was recorded at prediction time, reducing the risk of temporal leakage. It could also preserve request-scoped signals that might be impossible to recreate.

Everything sounds great with the above scheme, so why don't more teams adopt this approach? The tradeoff is time to get value from the feature. Teams have to invest production engineering effort to enable the low latency online feature before knowing whether the feature improved the model (Simha, 2020; Simha, 2023). An extreme example of this is for use cases which have seasonality such as vacation booking sites where travel patterns differ wildly between summer vs winter. Models requiring annual seasonality could need more than a year of logs, so a feature proposed today and implemented in the online system for data collection would remain un-incorporated in model training through several planning cycles.

Netflix reported the same delay for recommender models trained on several weeks of data. Under feature logging, an experiment with a modified encoder had to wait several weeks for a fresh dataset. It was reported that their fact store, Axion, could regenerate the feature from stored historical facts within hours (Kaushal, 2022). That difference changes the economics of experimentation because a weak idea can be discarded the same day, while a promising one can reach an online test when its context is still current.

The alternative was a manual backfill, which computes a newly defined feature over past data so that historical training examples become available immediately. A data scientist might write a warehouse query to calculate six months of values from raw events, avoiding the calendar delay while creating a second implementation beside the online computation.

Those implementations often use different languages and systems. Offline logic may run as SQL or Python in Spark, while online logic may run in Java, Go, or C++ and consume a stream processor or service API. Engineers then translate window boundaries, missing-value behavior, filters, and joins across both environments, bringing back the problem from Section 4.3 as two codebases drift while claiming to compute the same feature. And the situation gets worse because recsys models these days easily have 100s if not 1000s of features.

Airbnb described the resulting choice as a trade-off among iteration speed, engineering effort, and consistency. Manual backfills could take weeks or months across large historical datasets, and implementation discrepancies might remain invisible until production labels exposed degraded performance (Simha, 2020). GoJek faced the organizational version of the same issue when Python feature transformations were rewritten for online serving and new infrastructure was assembled for every deployment context (Lim, 2020).

The handoff between data science and engineering magnifies this delay. A data scientist can demonstrate that a feature works on the offline side in a notebook, although that notebook may assume BigQuery data access, flexible latency, and libraries unavailable in the serving environment. A

production engineer then has to interpret the prototype, reproduce the transformations, create ingestion jobs, define schemas, provision storage, add monitoring, and connect the feature to the model service.

Sculley and colleagues identified the structural cause in their account of technical debt in machine learning systems (Sculley et al., 2015). They described pipeline jungles built from accumulated scrapes, joins, and intermediate files, and connected them to overly separated research and engineering roles. They also named the multiple-language smell, where ownership transfer and testing become harder whenever a feature crosses language boundaries.

A more scalable approach treats a feature definition as an executable specification rather than a one-off query. The definition identifies source data, entity keys, event timestamps, transformations, aggregation windows, output type, and freshness requirements, after which a platform compiles or interprets it for two contexts. The offline context runs a scalable, point-in-time backfill over historical data, while the online context updates and materializes the same logical feature for serving.

For example, a definition might request the number of restaurant orders completed during the previous 30 minutes, grouped by restaurant. The historical executor computes that value at every training example's timestamp, and the streaming executor updates the aggregation as order events arrive. Both follow the same rules for filtering, window boundaries, and missing values, with the online store retaining the latest result for each restaurant while the offline store retains or reconstructs historical results.

Declarative systems may still use different execution code inside each engine, such as batch SQL for one path and a streaming operator for the other. The shared specification gives both engines one semantic contract, and consistency measurement verifies that they honor it. Platform engineers maintain translation and validation centrally so that each model team does not rebuild the machinery for every feature.

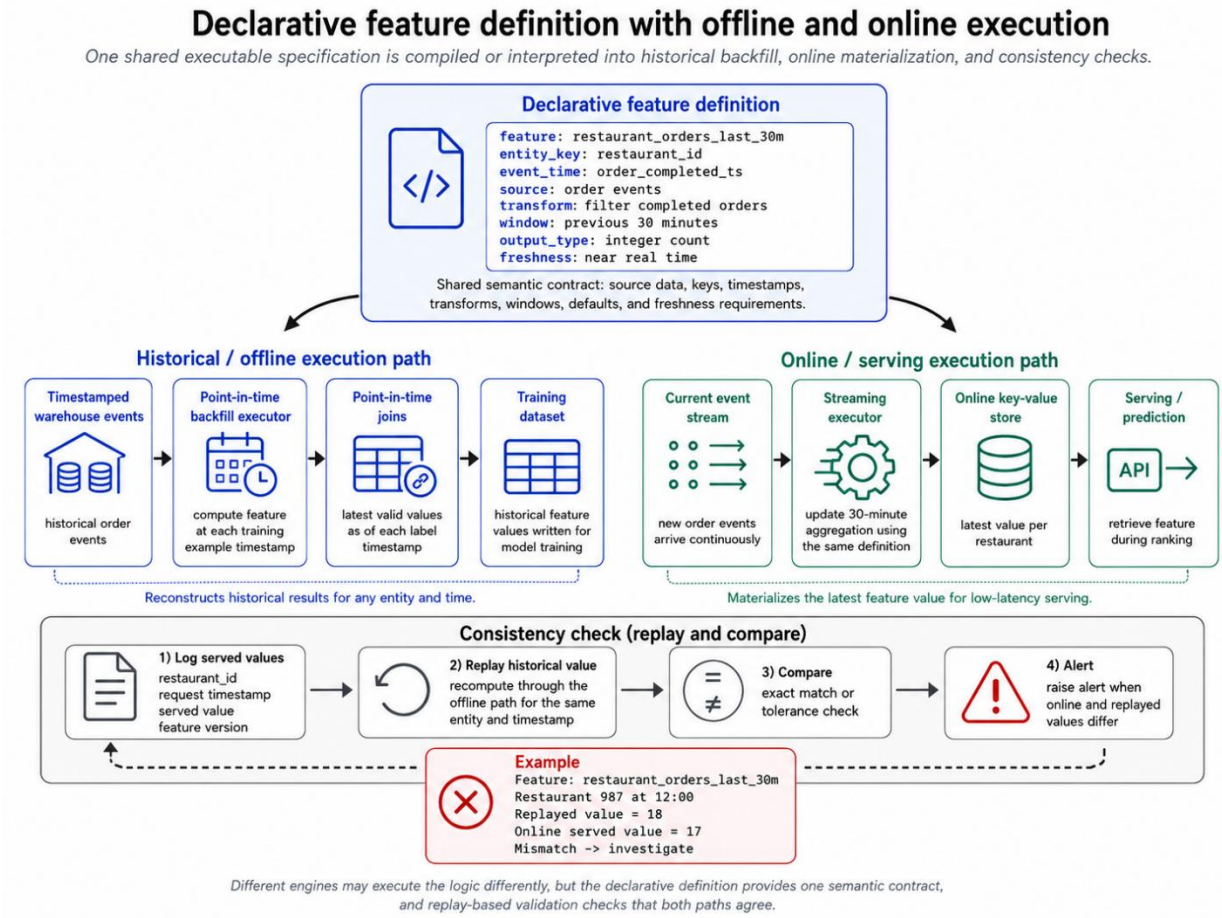


Figure 4.5: One feature definition drives historical backfills and online materialization, shortening experimentation while preserving a measurable consistency contract.

Airbnb's Zipline applied this pattern to sources, grouped aggregations, and temporal joins, generating the underlying offline and online pipelines from a Python specification. Work that had consumed months could be completed in days. Chronon, the later framework, reported that practitioners typically generated new feature sets in less than a week and had created more than ten thousand features at Airbnb (Simha, 2023; Airbnb Engineering, 2024).

LinkedIn achieved a similar result with Feathr. Teams declared anchored features tied to data sources and derived features built from other features, then Feathr replayed those definitions over historical time-series data for training and materialized values for inference. LinkedIn reported that adding and experimenting with features fell from weeks to days, and in some workloads the shared processing framework ran as much as 50% faster than the custom pipelines it replaced (LinkedIn Engineering, 2022).

DoorDash addressed the same bottleneck in stages. Its engineers initially wrote native Flink applications for real-time features, but repeated boilerplate and slow iteration led to Riviera, a declarative framework based on configuration and Flink SQL. Feature iteration fell from weeks to hours and codebase size declined by more than 70%. DoorDash later expanded the approach with Fabricator, which coordinated batch, streaming, registry, and storage workflows (DoorDash, 2021; DoorDash, 2022).

These systems still require substantial engineering. Someone has to define event-time semantics, operate compute engines, handle late data, evolve schemas, monitor freshness, control cost, and test online-offline agreement, but that work moves into reusable infrastructure and stable contracts. Model teams can concentrate on a feature's meaning and evidence of value without rebuilding its delivery system each time.

There are also limits to what the pattern can express. Some signals exist only during a live request, perhaps as a response from an external service, and these features may still require logging and waiting because no historical source can reproduce them. Other transformations depend on libraries or stateful logic outside a declarative framework's capabilities, requiring custom computation paths followed by explicit consistency tests.

We have now arrived at four pieces through operational necessity. Low-latency serving required an online key-value store; training-serving skew required shared definitions and continuous comparison; temporal leakage required timestamped history and point-in-time reconstruction; and slow iteration required automatic backfills from the same logical definition used online. The next step is to assemble these pieces and give the resulting system a name.

4.6 Convergence: The Feature Store Assembles Itself

By the end of the previous section, we have assembled four pieces of infrastructure that tend to be recognized in mature ML systems. A key-value serving layer puts precomputed feature values within the latency budget for online inference, while shared definitions reduce differences between training and serving. Temporal joins reconstruct the values available at each historical prediction time, and declarative backfills generate that history without requiring a second implementation. Once you add a registry for feature names, types, owners, and sources, the result has the shape of what we now call a feature store.

No universal top down specification produced this architecture. Several companies had likely arrived at similar designs after encountering local failures, before the wider industry had agreed on the category's name. Netflix described DeLorean in 2016, when “feature store” was not yet common terminology, as a system that captured historical snapshots of facts such as viewing history. It then ran the feature encoders used by online services to reconstruct past model inputs, subject to one memorable requirement: “no paradoxes are allowed.” In other words, a label or any other future information could never appear in the features (Netflix Technology Blog, 2016). Google’s Rules of Machine Learning, published during the same period, advised teams to log the features used at serving time, reuse code across training and serving, and measure training-serving skew explicitly (Zinkevich, 2017). Both organizations had named the core problems before the industry settled on a name for the combined solution.

Uber supplied that name in its 2017 account of Michelangelo, which included a centralized Feature Store where teams created, managed, and shared canonical features. Batch pipelines wrote feature history to the offline environment and materialized current values into Cassandra for low-latency access, while streaming pipelines supplied near-real-time values and logged them back to the offline environment for later training. Model configurations referred to features by canonical name, and a shared transformation language applied the same expressions during training and prediction. Uber reported that the store contained about 10,000 features at the time (Del Balso and Hermann, 2017).

Airbnb and LinkedIn reached closely related designs from different starting points. Airbnb’s Zipline generated point-in-time-correct backfills and online pipelines from declarative feature specifications, an

approach its successor Chronon retained while adding explicit consistency measurement. Chronon can replay logged online requests through a historical backfill and compare the reconstructed values with those actually served. LinkedIn's internal Frame system, later released as Feathr, provided a common namespace and replayed feature definitions over historical time-series data for training, then materialized the corresponding values for online inference. LinkedIn compared feature consumption to importing a software package: the consumer names a dependency, and the platform resolves how that dependency is sourced and computed (LinkedIn Engineering, 2022).

GoJek encountered the same pattern while operating Jaeger, its driver-allocation system, where Python feature transformations written for training had to be rewritten for online serving. The duplicated work consumed engineering time and introduced skew, while other teams built features for the same customer, driver, and geographic entities with no straightforward way for Jaeger's team to discover or reuse them. GoJek and Google Cloud responded by releasing Feast in 2019, which gave the emerging architecture an open-source implementation (Sell and Pienaar, 2019; Lim, 2020). Twitter later adopted and extended Feast, and its 2021 feature-store presentation ranked three goals in order: consistency, experimental and production velocity, and feature sharing (Liu, 2021). That ordering captures the broader convergence because a catalog was useful only after teams could trust the values it exposed.

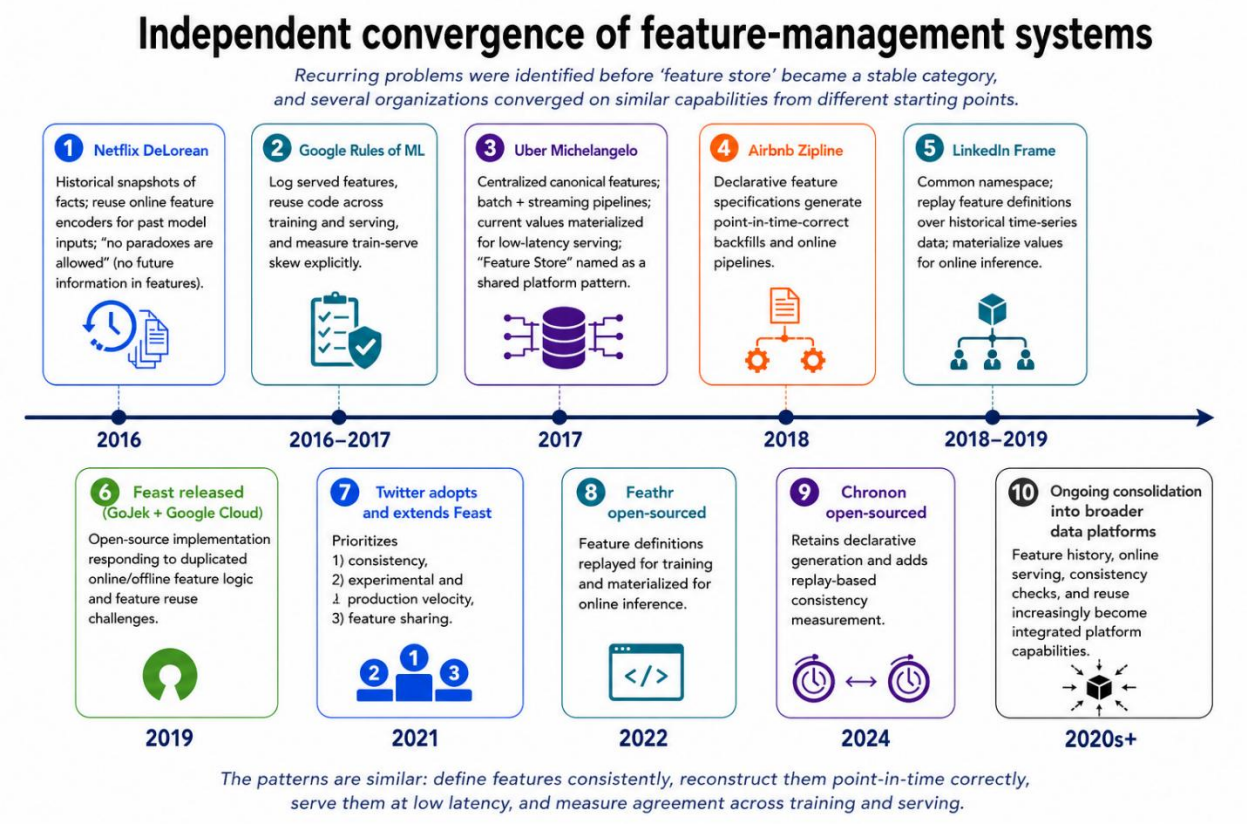


Figure 4.6: *Independent convergence of feature-management systems. Leading companies operating production ML systems across various domains identified recurring problems before "feature store" became a stable category and independently assembled similar capabilities from different starting points.*

Productization followed soon after these internal systems converged. Logical Clocks released the Hopsworks Feature Store as part of Hopsworks 0.8.0 in 2018, describing it as the first open-source feature store embedded in an enterprise data platform. Feast provided another open-source implementation in 2019, while Tecton took the pattern in a managed commercial direction. Founded in

2019 by engineers who had worked on Uber's Michelangelo platform, Tecton publicly launched in April 2020 and made its managed enterprise feature store generally available that December. These systems already represented different boundaries: Hopsworks integrated feature management into a broader data platform, Feast coordinated existing computation and storage infrastructure, and Tecton managed feature transformation and serving as a cloud service (Hopsworks, 2018; Sell and Pienaar, 2019; Tecton, 2020a; Tecton, 2020b).

The major cloud and data platforms adopted the pattern shortly afterward. Amazon launched SageMaker Feature Store in December 2020, Google introduced Vertex AI Feature Store in May 2021, and Databricks announced its Feature Store that same month before making it generally available in April 2022. Their arrival moved feature stores from specialized infrastructure into larger managed ML platforms, although it did not produce one standard architecture. Each vendor combined feature discovery, historical retrieval, online serving, transformation, and monitoring differently, which helps explain why products carrying the same label can have substantially different system boundaries (AWS, 2020; Google Cloud, 2021; Databricks, 2021; Databricks, 2022).

The term still hides important differences, so it helps to unpack the capabilities before discussing products. Chip Huyen separates feature-store functionality into feature management, feature computation, and feature consistency (Huyen, 2022). **Feature management** covers the registry, including names, schemas, owners, versions, lineage, documentation, and discovery. **Feature computation** includes the batch, streaming, and request-time processes that turn raw events into values, while **feature consistency** covers the mechanisms that make training and serving observe equivalent definitions and temporally valid values. A product may implement any one of these capabilities, or it may own all three.

Feast illustrates why this distinction is key when you are selecting an architecture for your recommender system. At the time of writing, it provided a registry, point-in-time historical retrieval, materialization into online stores, and one interface for offline and online access, while leaving general-purpose transformation and workflow orchestration upstream. Teams commonly use warehouse SQL, dbt, Spark, Flink, or another processing system to compute the values that Feast manages and serves. Feast also relies on existing databases, such as a warehouse for history and Redis, DynamoDB, or another key-value system for online values. Its documentation explicitly says that Feast is neither an ETL system, an orchestrator, a warehouse, nor a database (Feast, 2026). Calling Feast a feature store is accurate within the category's broad usage as long as its boundary is understood: it supplies management and consistency around infrastructure the team already operates, without owning the complete feature-computation lifecycle.

Other systems draw a wider boundary. Feathr and Chronon execute substantial parts of feature computation, and some commercial platforms combine transformation, orchestration, storage, serving, and monitoring. Khadder describes these designs as literal, physical, and virtual feature stores: one manages computed values, another owns computation and storage, and the third coordinates existing infrastructure (Khadder, 2021). This taxonomy explains why two teams using the same label may operate systems with very different responsibilities.

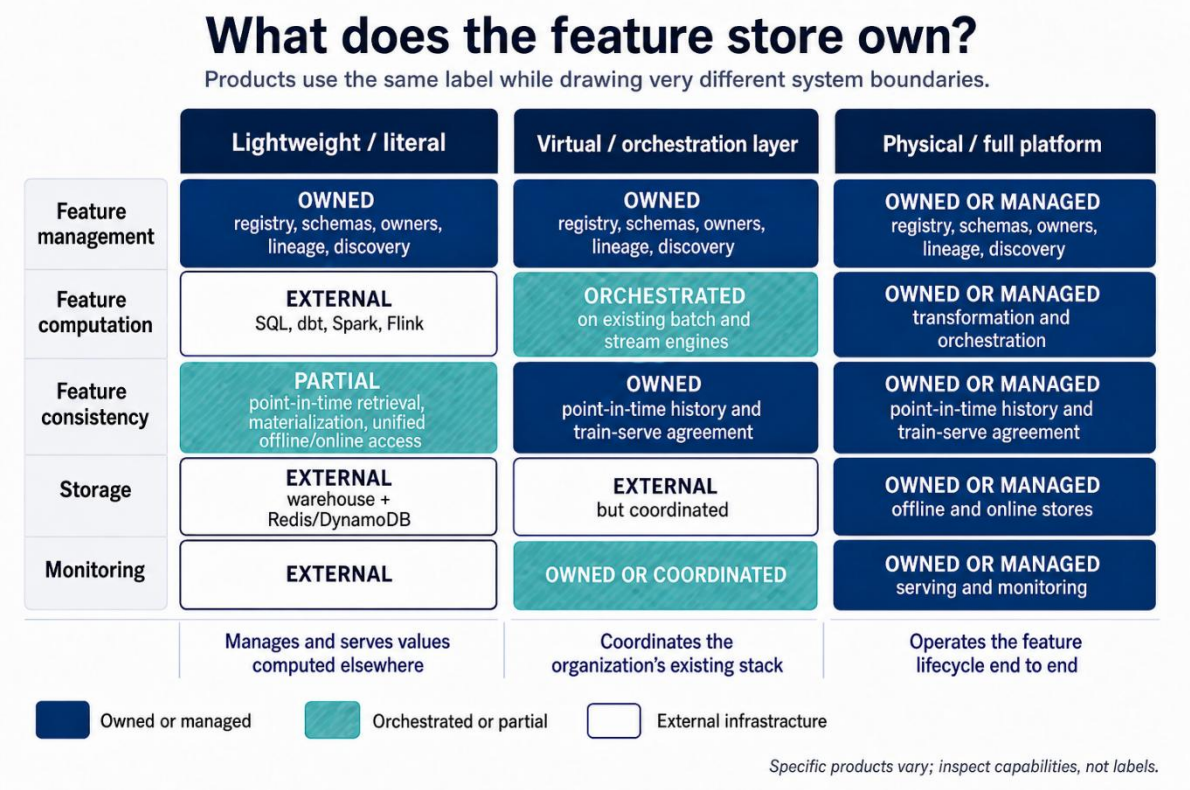


Figure 4.7: Feature-store products can draw their system boundaries around different combinations of management, computation, consistency, storage, and monitoring. The label alone does not reveal which capabilities a team is adopting.

You can also reason about the boundary through the three pipelines in an ML application. Hopsworks uses this decomposition to clarify where transformations belong and how values move between offline and online environments (Dowling and Hopsworks, 2024). Feature pipelines turn source data into reusable values, training pipelines combine those values with labels and fit models, and inference pipelines retrieve current values and execute predictions. Model-dependent normalization often belongs with the trained model because its parameters were learned from a particular training set, whereas a reusable count of a user's purchases over seven days belongs naturally in a feature pipeline. A query-item similarity (e.g., vector search cosine distance) that depends on the current request may need to run inside the inference path.

Teams can build the platform, adopt an open-source layer, buy a managed system, or assemble the required capabilities from their existing data stack. DoorDash, Netflix, Uber, and Airbnb built around their own scale and workflows, while Shopify built Pano on Feast. Robinhood's public talks describe adopting Feast and later rewriting the system as its requirements expanded, and Spotify's Home surface once operated five feature stores before moving toward a unified platform. These histories are useful reminders that the right decision on buy vs build depends on a complex set of enterprise capabilities.

The most useful mental model is therefore architectural. A feature store is a coordinated set of capabilities that manages feature definitions, produces or receives values, reconstructs valid history, and delivers consistent inputs to training and inference. Storage is one component, and the organizational contract around definitions, ownership, and reliability carries equal weight. Once you unpack the term this way, you can ask a more precise adoption question.

4.7 Do You Actually Need a Feature Store?

Industry convergence does not make a feature store a universal requirement. Huyen surveyed 95 companies in the early 2020s and found that about 40% used one, with half of those users having built their own (Huyen, 2022). A Feature Store Summit poll found that 50% favored building, 35% favored buying, and 15% remained undecided (Redis, 2021). Both samples were small and self-selected, so they are better treated as signals than as market censuses, yet they still challenge the assumption that every production ML system starts with a feature store.

Begin the decision with two separate questions. First, does the organization have a **feature-management problem**, visible through incompatible definitions for the same concept, repeated feature work, unclear ownership, missing lineage, or difficulty discovering existing features? Second, does it have a **feature-serving problem**, such as the need for fresh values at low latency, repeated construction of online databases and ingestion jobs, error-prone historical joins, or persistent differences between training and serving? A full feature platform earns its operational cost most clearly when both groups of problems are already present.

Small scale ML projects often have neither problem. A team operating one batch model can generate a feature table in its warehouse, score the relevant entities on a schedule, and write predictions to an application database or cache. The warehouse already provides historical storage, access controls, and transformation tooling, while a versioned SQL project can hold the feature logic in a form that another engineer can review. Add tests for schemas, null rates, value ranges, and temporal predicates, and the team has covered the immediate reliability needs without introducing another service to deploy and monitor. A premature push to develop a dedicated feature registry, online store, materialization service, and separate operational control plane would increase the failure surface without removing a demonstrated utility.

Request-scoped features provide another case where a store may contribute little. The current query language, device type, shopping-cart contents, or similarity between query and candidate embeddings can all be available within the request. Such features may benefit from a shared transformation library, although there may be no entity value worth materializing ahead of time. Their computation belongs on the read path, the portion of the system that runs when the model receives a request, and an additional store can simply add a network hop.

Monzo's feature system shows how narrow the solution can remain even after a serving problem appears. The bank already held current account balances in production, while its models also needed slow-changing analytical aggregates, such as a customer's average balance over seven days, that existed only in BigQuery. Early services queried BigQuery directly and cached the results. Once the pattern began repeating, Monzo built a small bridge: data scientists defined tested SQL feature tables, scheduled jobs exported the results, and a synchronization process wrote them to Cassandra. A Go service returned each value together with its last-updated metadata. The system did not attempt general streaming computation or a comprehensive training-data API because the goal was simply to move a subset of numbers from analytics into production while spending little time on infrastructure (Lathia, 2020; Monzo, 2022).

Monzo also provides a useful challenge to the shared-code principle. An argument can be made that independent SQL and Go implementations can make different classes of mistakes, which produces two outputs that teams can reconcile. This design accepts duplication deliberately and uses comparison as a control, an approach that can work when the online logic is simple enough to reproduce and values can be checked continuously. It does not generalize well to complex windowed aggregates spread across

hundreds of features, where implementation costs grow and reconciliation itself requires platform support. The important lesson is that identical code is one route to consistency, while systematic measurement offers another.

Training-serving skew has other smaller remedies as well. The risk appears when training and serving compute supposedly identical features through divergent logic or data, and a shared, versioned module can remove that divergence for one team. Logging the exact feature values used during inference, then training from those logs, can provide an even stronger guarantee at the cost of waiting for sufficient history. Google recommended this pattern in Rule 29, and YouTube's home page reported improved quality and lower code complexity after adopting it (Zinkevich, 2017). Neither approach requires a feature-store product.

Point-in-time correctness can remain in the warehouse while scale is modest. For each labeled example, a temporal join selects the latest feature record that was available by the prediction timestamp, and a small team can review and test those queries directly. Specialized backfill engines begin to justify their complexity when examples require many windowed aggregates across billions of events, or when multiple teams repeatedly implement the same temporal logic.

Operational burden deserves the same scrutiny as capability. Feast can reduce custom serving code, although the team still owns upstream jobs, orchestration, offline and online storage, and all the connections among them. A managed platform transfers some of that work to a vendor while introducing integration, cost, and dependency trade-offs. Public, named postmortems from abandoned feature store integrations are scarce, so anonymous accounts should not determine the decision. The stronger first-party histories point to a narrower conclusion: system boundaries need to match local requirements.

The costs already visible give you a practical threshold. Defer the decision on building/buying a dedicated feature store when several of these conditions are true:

- predictions tolerate batch processing,
- one team owns a small number of models,
- feature reuse is uncommon,
- request-time features dominate,
- and warehouse queries already satisfy latency and history requirements.

Start with versioned transformations, explicit owners, schema and freshness tests, plus a cache or key-value table only where serving latency demands it, then revisit the decision when those components begin repeating across teams. That foundation is already a lightweight feature-management system, and its components can later move behind a registry or unified API without discarding the underlying feature logic.

Adoption becomes easier to justify when several teams share features across many models, online inference needs low-latency values from batch and streaming sources, and engineers repeatedly build materialization or backfill pipelines. Difficult-to-audit temporal joins and governance requirements for searchable ownership and lineage are additional signals. Even then, evaluate the three capabilities from Section 4.6 independently because a team may need a registry without another database, or point-in-time retrieval without streaming computation. Buying the entire category to fill one capability gap creates avoidable complexity.

Treat the decision as a boundary-setting exercise: identify the concrete failure, choose the smallest capability that removes it, and preserve a path for expansion. This discipline also prepares the team for a separate question, which is how fresh each feature and model needs to be.

4.8 Fresh Features, Fresh Models: The Freshness Dividend

A re-ranking model can serve predictions in 20 milliseconds while still responding to the prior week's stale view of the world. The term 'Latency' alone is not sufficient to tell you whether the decision uses current information. Model **serving latency** includes the time needed to fetch features and produce a prediction after a request arrives. **Feature freshness** measures the age of the source information represented by those features, while **model freshness** captures the delay between new behavior entering the learning process and its influence on the labels reaching the deployed model. These three can interact, although each measures a different source of delay.

Consider an item-availability feature refreshed every two hours. Redis may return the stored value in one millisecond, which gives excellent serving latency even though the value can be nearly two hours old. A model retrained weekly may then interpret that stale value using parameters learned before the latest changes in demand. Faster retrieval repairs neither source of staleness.

ByteDance quantified the value of model freshness in Monolith, its online training system for recommendation. Monolith joins actions with features from Kafka streams, updates model parameters, and synchronizes changed parameters to serving machines. In a streaming simulation on the Criteo dataset, online-serving AUC increased monotonically as the parameter synchronization interval shortened from five hours to one hour and then to 30 minutes. In a production advertising experiment, online training exceeded batch training's serving AUC by 14.4 to 18.1 percent on each of seven reported days (Liu et al., 2022). These relative improvements belong to the Monolith authors' models and experiment, which makes them evidence of a freshness effect in that setting rather than a forecast that should be transferred directly to every recommender.

Monolith also demonstrates why freshness policies should vary by parameter type. Sparse parameters include embeddings associated with individual users, items, or categorical identifiers, and only a fraction of those identifiers receive updates during a short interval. The system can therefore synchronize touched entries frequently. Dense parameters influence the model more broadly, which makes continuous replacement more expensive and riskier. ByteDance synchronized sparse parameters at minute-level frequency while updating dense parameters daily, concentrating bandwidth where recent interactions were most likely to change a recommendation.

Grubhub found a similarly strong cadence effect in food recommendation. Using no retraining as the baseline, weekly retraining increased *purchase-through rate*[2] by 2.5%, while daily retraining increased it by 20.3%. The team attributed the difference to drift in customer preferences, restaurant conditions, and marketplace behavior. Its stateful incremental-learning design preserved model state between updates, achieved about a 20% increase over the baseline, and reduced training cost by a reported factor of 45x (Egg, 2021). Because each run continued from the current model instead of rebuilding the complete state, stateful updating changed the economics of freshness.

[2] Egg reports Purchase Through Rate as Grubhub's online business metric measuring the rate at which recommendation exposure results in a purchase. The paper reports relative changes in this metric but does not disclose its exact denominator or attribution rules (i.e., whether the denominator is recommendation impressions vs sessions, etc.)

Fresh features can create value even when model parameters update less often. Instacart's initial batch-oriented system could not represent a customer's current shopping intent or current product availability, so its real-time platform moved availability updates from hours to seconds and supplied session behavior through Kafka and Flink. Subsequently, Instacart reported better *found* rates, fewer bad orders, measurable growth in gross transaction value across experiments, and millions of dollars in annual fraud-cost reductions across the platform's use cases (Instacart, 2022). The source does not isolate one universal effect size for feature freshness, although it does connect fresher availability and session signals to concrete product outcomes.

These cases suggest a hierarchy rather than one platform-wide target. Illustrating the differences in an ecommerce case: Request context, such as the current query or cart, is created at inference time, while session features summarize the current visit and may require updates with seconds to sub-second freshness. Inventory levels/item availability, may tolerate several seconds to low minutes depending on the use case. Longer-term preference aggregates can often refresh hourly or daily, and stable attributes such as a product category need no update until the source record changes. One freshness target wastes resources on slow-changing features while underserving volatile ones.

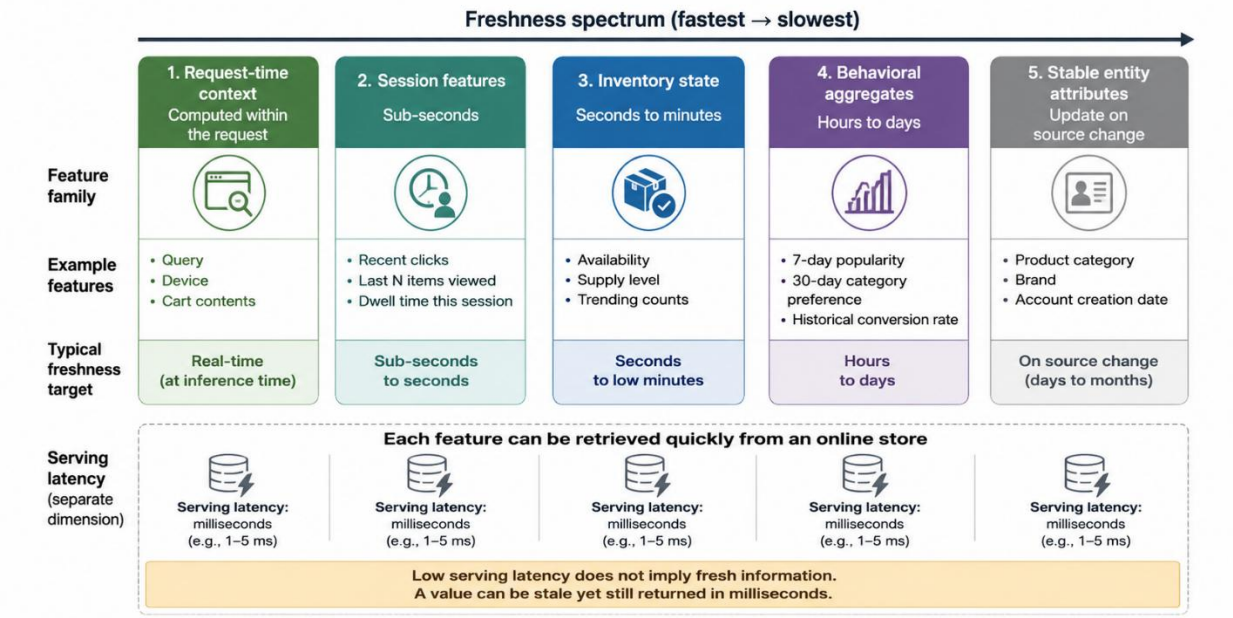


Figure 4.8: Different feature families require different freshness objectives. Serving latency remains a separate property because a stale value can still be returned quickly.

[3]Each feature should have a **freshness objective**, meaning a stated upper bound on how old its value may be for the product decision and a threshold the owning team can defend operationally. You also need an observable timestamp to enforce that objective instead of assuming that a successful pipeline run implies fresh data. Ingestion time alone can hide upstream delays, so retain the event time when the underlying behavior occurred, the computation time when the feature was produced, and the materialization time when it reached the serving store. Differences among those timestamps locate the delay, since a newly materialized value derived from an old source event remains stale from the model's perspective.

[3] You might ask, "Why bother?" Why not just ask your Data Engineering team to enable every feature update with sub-second freshness SLAs? Reason is the exponential increase in the costs (compute & storage) as well as increasing surface area of errors.

First-generation feature stores commonly improve retrieval by precomputing values and materializing them into an online store. This write-path design moves computation ahead of the request and produces predictable serving latency, while the upstream pipeline determines its freshness ceiling.

Compute-on-read moves selected work into the request path. Fennel frames this choice as a trade-off between the **write path**, where events are processed and values prepared, and the **read path**, where a prediction request retrieves or computes what it needs (Fennel, 2023). Write-path computation suits expensive aggregates reused across many requests, whereas read-path computation suits request-specific logic, sparse access patterns, and values that must reflect the latest source state. Hybrid designs precompute stable intermediate results and combine them with current context during the request, which turns materialization into a per-feature optimization governed by cost, latency, and freshness.

Greater freshness also expands the system's failure modes. Streaming joins have to handle late and out-of-order events, online training can amplify corrupted feedback quickly, and frequently synchronized models need rollback points and validation gates. ByteDance accepted different staleness for sparse and dense parameters and documented a bounded loss of recent updates after certain failures, reflecting an explicit reliability trade-off. A system updating every minute without adequate monitoring can spread a bad feature or label hundreds of times faster than a daily pipeline.

The freshness dividend is therefore conditional. Faster updates create value when the environment changes quickly, recent behavior predicts the next decision, and the system can validate and recover at the same cadence. Measure business response across realistic refresh intervals, then assign different service objectives to different features and parameter groups. The next section covers the control system required to make those faster pipelines trustworthy.

4.9 Silently Wrong Is Worse Than Loudly Broken

Uber once discovered data quality issues in its fares data 45 days after it began, and the problem was found manually. After the company introduced its D3 automated drift-detection system, average detection time for covered datasets reportedly fell to 2 days. D3 monitored over 300 high-priority datasets and reported 95.23 percent detection accuracy on fact tables (Shukla et al., 2023). The specific numbers belong to Uber's enterprise environment, but have been quoted to illustrate a general danger: a data pipeline can complete successfully, publish on schedule, and provide functionally wrong values to the model.

A loud failure stops a job, times out a request, or raises an exception, all of which existing operational systems detect reasonably well. A silent failure preserves the expected shape while changing the meaning. An event producer can stop emitting one category and lower a count, a timestamp can change units, or a default can replace a missing field. A join may still return rows after an identifier migration while matching the wrong entities. In every case, the model continues accepting numeric tensors and producing scores.

Netflix describes the consequence as a loss of trust. Its Axion fact store supplies historical inputs from which recommendation features can be regenerated, yet a researcher does not care whether Axion or an upstream component originally corrupted the data. If bad data arrives through Axion, trust in Axion declines (Kaushal, 2022). Rebuilding that trust is difficult because the visible symptom may appear only after labels arrive, an experiment loses, or the customer experience degrades.

Feature reliability starts before deployment with a **data contract**, which records the assumptions a consumer makes about a feature or source. The contract should cover schema, type, entity key, timestamp semantics, allowed values, null policy, freshness objective, owner, and compatibility rules.

Tests can then evaluate transformations against small, controlled inputs. For example, a seven-day purchase count should exclude events after prediction time, handle a user with no history, define whether interval endpoints are included, and produce known results for late or duplicated events. These checks test the feature's semantics rather than merely proving that its pipeline executed.

The ML Test Score treats feature code as production code. Its proposed checks include capturing feature expectations in a schema, testing all feature code, notifying consumers when dependencies change, measuring training-serving skew, and monitoring staleness (Breck et al., 2017). The minimum score across data, model, infrastructure, and monitoring tests determines the overall result, which reflects how one weak layer can compromise the system.

Pre-deployment validation should operate at several levels. Schema and constraint tests detect missing fields, invalid ranges, and unexpected nulls, while transformation and temporal tests exercise edge cases and prevent access to future values. Reconciliation tests compare computation paths, and integration tests verify that keys, timestamps, and serialization survive the full journey through offline and online environments.

Production monitoring[4] asks whether values arriving now remain consistent with the contract and with recent history. **Freshness monitoring** measures the age of the latest valid value, and **volume monitoring** tracks row, event, and entity counts. **Distribution monitoring** follows statistics such as missingness, quantiles, category frequencies, and embedding norms, while **skew monitoring** compares features observed during training and serving. **Lineage monitoring** connects an anomaly to upstream jobs and downstream models, allowing responders to estimate its impact.

Static thresholds work well for firm invariants, such as a probability that falls outside zero to one, but other signals vary by hour, weekday, geography, or season. Uber's monitoring systems modeled historical patterns and automated drift detection across many columns (Uber Engineering, n.d.). Statistical detection complements domain constraints, although owners still need lineage and context to distinguish corruption from a product launch, holiday, or genuine change in behavior.

Train-serve comparison deserves special attention because each side can look reasonable when inspected independently. Google Play found features that were always present in training and always missing from serving logs, a mismatch that neither dataset's schema necessarily exposed. Comparing same-day statistics across the two contexts revealed the problem, and removing the skew improved installs on the main Play landing page by 2 percent in an online experiment (Baylor et al., 2017). Chronon takes a more direct approach by recording online fetches, reconstructing corresponding values through its offline backfill, and diffing the results. That comparison directly tests the consistency promise the platform was built to make.

Alerts also need operational meaning. Every production feature should ideally have an owner, a severity policy, and a documented response that reflects the risk of using an invalid value, so the on-call engineer does not have to invent[5] policy during an incident. A stale recommendation count might fall back to a longer-term aggregate, while a missing fraud signal could require failing closed or routing the case to manual review. Removing a corrupted feature from model input is safe only when the model and serving code define appropriate defaults and those defaults have been tested before an emergency. Depending on the incident, rollback may involve the feature definition, its materialized values, the model, or all three. Monitoring without a response path produces dashboards rather than reliability.

[4] We will cover Ops in later chapters.
[5] Though this is more often the case if your team is launching online recsys models for the first time.

The tradeoff with faster pipelines is that they leave less time for detection. A daily batch job may allow hours of validation before publication, while a streaming feature can reach millions of requests within minutes and an online learner can absorb it into model parameters. Validation therefore moves to both sides of deployment: unit and contract tests run before release, while online checks, canaries, and reconciliation continue afterward. The release process should constrain the initial blast radius and preserve the last known good feature values and model state.

The goal is observable correctness. Teams should be able to identify which source event produced a value, which definition version transformed it, when it was materialized, which models consumed it, and whether online and historical paths agreed. A system that answers those questions turns a vague decline in model quality into an incident that engineers can locate and repair.

4.10 Ten Versions of the Same Feature

Uber's Palette team found ten different versions of a feature representing the trips a driver took each day. Every version consumed compute and maintenance effort, while models that appeared to use the same business concept could operate on different values (Nene and Chen, 2019). This duplication revealed missing organizational memory: teams could create features, yet they could not reliably discover, evaluate, and depend on work produced elsewhere.

GoJek saw the same problem around Jaeger. Most dispatch features described customers, drivers, or geographic areas, and their internal project teams were already building features for those entities. Jaeger's team had no easy way to discover or integrate them without reimplementation (Lim, 2020). LinkedIn described the underlying infrastructure gap more formally: team-specific pipelines had no uniform names, type system, deployment process, or serving mechanism, which made sharing prohibitively difficult (LinkedIn Engineering, 2022).

A **feature registry** mitigates[6] this gap by giving each managed feature a stable identity and searchable metadata. An entry should include the definition, entity key, type, source, owner, version, freshness objective, lineage, validation status, and known consumers. You can think of the registry as serving a role similar to a package index in software engineering. A model declares the features it consumes by name, the platform resolves their definitions and dependencies, and consumers inspect each contract before adopting a feature that its producer maintains.

Names[7] alone do not create reuse because two daily-trip features may encode legitimately different concepts. One might count completed trips in UTC, another accepted trips during a driver's local day, and a third might exclude suspected fraud. The registry should preserve those semantics instead of forcing a superficial merge. Canonicalization [8]requires domain review: identify the business concept, make window and filtering rules explicit, then decide whether variants are versions, parameters, or genuinely different features. Usage counts and lineage help reviewers determine which definitions have earned trust and which can be retired.

Versioning protects existing consumers as definitions evolve. Changing a seven-day window to a 30-day window changes the feature's meaning and should create a new name or version, whereas a backward-compatible documentation correction does not require one. Schema changes, source migrations, late-event policies, and category mappings all need explicit compatibility rules. Deprecation then becomes a

[6] Teams can still decide to not use an existing feature. Common reasons include they want to do the feature engineering slightly differently or they don't want dependency on other teams.
[7] Any documentation can abstract away the nuances and therefore the ultimate source of truth is always the source code.
[8] This is often a thankless job because the benefits are diffused both in space and time, while the costs are concentrated upfront.

managed process in which the owner identifies consumers, announces the replacement, compares values, migrates models, and removes the old pipeline after usage reaches zero.

The registry also requires ownership because an entry without an accountable team becomes dangerous as soon as its source changes. Ownership gives consumers someone who can interpret anomalies, approve semantic changes, and maintain the pipeline, while lineage connects the owned feature to upstream tables and downstream models. When a source column changes, the platform can notify affected owners and consumers before the change propagates silently.

Centralization creates a second-order problem of its own. Monzo observed that a central list becomes difficult to navigate once it contains thousands of features (Lathia, 2020). Uber's Palette eventually hosted hundreds of tables with as many as a thousand features in each, prompting Uber to build an internal feature-selection tooling that could identify redundant or weak signals (Uber Engineering, n.d.). The feature store reduced local duplication while creating a large discovery space. Search, documentation quality, usage telemetry, freshness status, and deprecation workflows determine whether that space behaves like a useful package ecosystem or an abandoned artifact repository.

The incentive structure is part of the system as well. Sambasivan and colleagues interviewed practitioners working on high-stakes AI and found that 92 percent had experienced at least one data cascade, defined as compounding downstream effects from data problems. Participants described data work as time-consuming, hard to track, and less rewarded than model work, with one summarizing the culture as, "Everyone wants to do the model work, not the data work" (Sambasivan et al., 2021). A registry asks teams to document, test, own, and support features for other users, so that labor needs to appear in planning, performance expectations, and platform metrics.

Governance metrics should extend beyond the number of registered features. Track active ownership, contracts, freshness objectives, adoption time, duplicates retired, and stale features removed, since a smaller catalog of maintained features creates more value than a large catalog of one-off experiments.

The ten versions of driver trips do not imply that every organization needs a large platform. They show that feature definitions had become shared production dependencies without the controls routinely applied to software dependencies. The appropriate remedy combines a registry with semantic stewardship, versioning, lineage, ownership, and incentives. Storage and serving make feature values available, while governance makes those values safe to reuse.

4.11 From Fixes to Platform: A Feature-Management Maturity Ladder

The systems described in this chapter generally began as responses to failures that engineers could see, measure, and explain. When warehouse retrieval missed a ranking request's latency budget, a team materialized selected values in a key-value store. When two implementations of a transformation drifted apart, the team shared the definition or trained from logged serving features. Historical joins that exposed future information led to point-in-time reconstruction, while repeated work across teams eventually gave features stable names, owners, and a registry. Each addition solved an observed problem, so the platform grew one capability at a time.

This progression gives us a useful framework for deciding what to build next, especially because feature-store vendor products draw their boundaries differently. The maturity ladder in Figure 4.9 is adapted from Eugene Yan's "Feature Stores: A Hierarchy of Needs" (Yan, 2021), which organizes capabilities around access, serving, integrity, convenience, and autopilot. Using the problem-derived language from this chapter, the ladder describes five cumulative outcomes: make features available; make them fast and fresh; make them correct; make them reusable; and make them self-operating. The relative priority

as indicated by its place in the hierarchy is important because a catalog has limited value when production models cannot retrieve its features, while automated backfills can spread mistakes when point-in-time correctness remains unresolved.

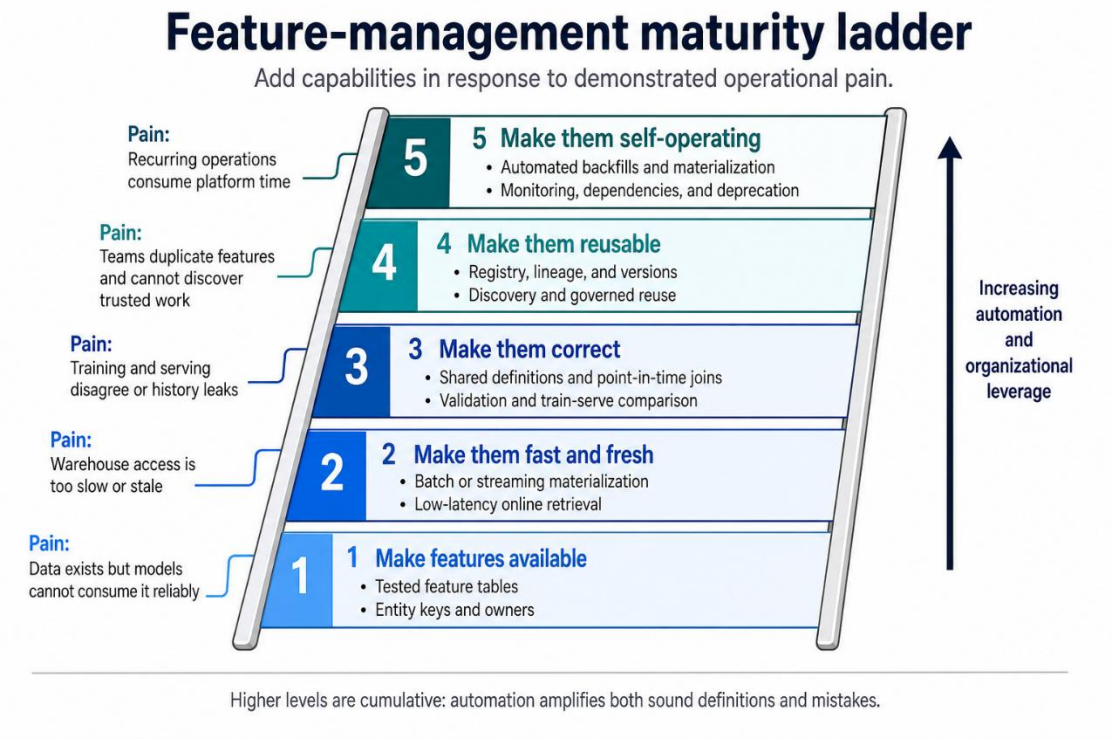


Figure 4.9: A problem-derived feature-management maturity ladder. Adapted from Yan, "Feature Stores: A Hierarchy of Needs" (Yan, 2021).

In the following sections, we will briefly describe each level.

Level 1: Make features available

The first level closes the gap between data that exists in the organization and data that a model can consume reliably. A useful signal may be buried in an event log, operational database, or warehouse table designed primarily for analytics. Availability means producing the feature repeatedly, associating it with the correct entity, and delivering it in a form that training and inference paths can read at the appropriate SLAs.

This first implementation can remain deliberately small because the immediate goal is dependable access. Monzo, for example, needed to move selected aggregates from BigQuery into Cassandra with freshness checks and validation around the transfer (Lathia, 2020; Monzo, 2022). A team running batch predictions may need only tested SQL, a materialized table, a documented owner, and a schedule aligned with scoring. You have reached this level when the model receives its features without someone copying files, rerunning an undocumented notebook, or reconstructing business logic from memory.

Level 2: Make them fast and fresh

Once a feature is available, its delivery pattern has to match the model's deployment pattern and the time available to make it. Online recommendation commonly requires low-latency reads across many candidates, which usually rules out warehouse queries inside the request path. Etsy described fetching

hundreds of features for roughly a thousand candidate listings per search-ranking request, while DoorDash reported tens of millions of feature reads per second (Walecka and Gallatin, 2023; DoorDash, 2020). These workloads lead teams toward precomputation, batched retrieval, and online key-value stores with predictable latency.

Speed and freshness are separate requirements because a store can return yesterday's value in a millisecond and satisfy only the first requirement. Freshness describes how closely a value reflects the current user, item, or environment, and the acceptable delay depends on how quickly that state changes. Batch updates may suit a stable product attribute, while session intent, item availability, or recent engagement may require streaming or request-time computation. Monolith and Grubhub show that model-update cadence can also affect recommendation quality, so the design should consider feature and parameter freshness together (Egg, 2021; Liu et al., 2022). Assign an objective to each feature instead of sending every signal through the fastest and most expensive path.

Level 3: Make them correct

Fast retrieval has little value when training and serving disagree, so correctness is where the platform begins to establish trust. The same feature name must carry the same semantics in training and inference, and every historical row must use information available at its prediction timestamp. The system must measure both properties continuously because late events, source changes, and duplicated transformations make them difficult to preserve as models multiply.

Google's Rules of Machine Learning recommends logging serving-time features, reusing transformation code, and measuring training-serving skew directly (Zinkevich, 2017). Netflix pursued the same outcome by storing historical facts and applying shared encoders through DeLorean and later Axion (Netflix Technology Blog, 2016; Kaushal, 2022). Airbnb's Zipline and Chronon took a declarative route, expressing feature logic once and replaying it over historical data with temporal joins (Zanoyan and Shapiro, 2018; Simha, 2023). These implementations differ, but they enforce the same contract: reproduce a feature value for a named entity at a stated time.

Correctness requires evidence before and after deployment because many failures remain invisible when teams check only whether a pipeline completed. Schemas, null behavior, ranges, timestamps, and transformation logic can be tested before release; drift, stale tables, missing values, and online-offline differences require production monitoring. A dependable implementation treats skew and freshness as service health signals alongside latency and availability, with owners who know how to respond (Breck et al., 2017).

Level 4: Make them reusable

Correct features become organizational assets when another team can discover them, understand their semantics, and adopt them safely. Central storage alone cannot provide that confidence because a reusable feature needs a stable name, definition, entity key, data type, owner, lineage, freshness expectation, and deprecation policy. Consumers also need enough evidence to decide whether its semantics fit their model.

LinkedIn compared Feathr to a package manager: a model names its features, while the platform resolves how they are sourced and computed (LinkedIn Engineering, 2022). GoJek built Feast after teams repeatedly created features for the same customer, driver, and geographic entities without a shared discovery path (Lim, 2020). Uber found several versions of similar features and later had to improve discovery within its growing catalog (Nene and Chen, 2019). A registry helps only while

ownership, review, and deprecation keep its contents trustworthy; otherwise, duplication simply acquires official names.

Reuse should follow demonstrated overlap, particularly when one team owns only a few production models. That team may gain more from a shared, versioned feature module than from operating a central registry whose workflows nobody else uses. As models, teams, and regulated dependencies multiply, common definitions and lineage create increasing leverage, and a collection of pipelines begins to function as a feature-management platform.

Level 5: Make them self-operating

The final level automates recurring work that otherwise consumes platform and model-development time whenever a feature changes. From a feature definition, a self-operating system can generate point-in-time backfills, materialize values, validate schemas and distributions, compare online values with historical reconstruction, alert owners, manage dependencies, and support safe deprecation. Airbnb's Chronon generates historical and online computations from declarative definitions, then checks consistency by replaying logged online requests (Simha, 2023). DoorDash's declarative frameworks similarly reduced the need to build and operate separate batch and streaming pipelines by hand (DoorDash, 2021; DoorDash, 2022).

Automation increases the return on a sound abstraction, and it also lets a faulty definition travel farther before someone notices. The same error may propagate through training sets, online stores, and dependent models, so self-service depends on Level 3 correctness contracts and Level 4 ownership. A mature automated system still needs escape hatches for request-context features, third-party responses, and unusual computation that does not fit the common path.

Use the ladder as a diagnostic for choosing the next capability that an ML organization can justify. If tested warehouse tables satisfy batch models, Level 1 may be sufficient; if online retrieval is the only constraint, a materialization job and key-value store may cover Levels 1 and 2. Persistent skew, expensive temporal joins, repeated definitions, or growing operational burden can justify later levels, but each step should solve an observed problem. Axion offers the pragmatic lesson: begin with a simple design, observe where it fails, and add machinery in response to evidence (Kaushal, 2022).

The ladder also separates platform maturity from product selection because no purchasing decision determines how well a team manages features. Warehouse SQL, a shared library, a cache, and disciplined monitoring can satisfy several levels when their operational contracts are clear. Feast relies on upstream systems for transformation and orchestration while providing registry, historical retrieval, and serving capabilities (Feast, 2026). Chip Huyen's division into feature management, feature computation, and feature consistency explains why products with the same label cover different parts of the ladder (Huyen, 2022). The appropriate endpoint is the smallest dependable system that resolves recurring failures and can expand when a new constraint appears.

4.12 Summary

Feature management work deserves the same engineering care as model code because a sophisticated ranker cannot recover information that arrived late, was joined from the future, changed meaning, or silently disappeared. Reliable personalization depends on the pipelines, definitions, tests, stores, and ownership practices that make every model input trustworthy in production.

Production feature management grows from constraints that become visible as models move from a notebook into a serving system. Request-time latency constraints imposes system-level design choices

towards precomputation and online retrieval, while separate training and serving paths allow the same feature to acquire different meanings. Historical reconstruction during the training dataset creation, introduces point-in-time joins or replayable facts, slow backfills motivate shared definitions, and growth across teams creates pressure for discovery, ownership, and reuse. Freshness and data quality remain operational responsibilities because a pipeline can run on schedule while delivering stale or semantically incorrect values.

Engineering leadership within your company may observe these ad-hoc fixes and may converge on the need to onboard a feature platform, although the label says little about where the system's boundary should sit. If you are in a position to own or influence the creation/selection of this feature platform, then evaluate the capabilities separately: managing definitions, computing values, preserving training-serving consistency, and delivering features at the latency and freshness the product requires. A small team can assemble only the pieces its models need, while a larger ML organization may justify registries, automated backfills, streaming computation, multi-store materialization, and continuous validation.

The maturity ladder provides a final checkpoint: make features available, fast and fresh, correct, reusable, and eventually self-operating. Progress should follow measured pain because the platforms discussed in this chapter grew after engineers encountered specific failures, and several were rewritten as those failures changed. Before adding another layer, identify which internal team/developer archetype experiences the problem, how often it has occurred, and whether a smaller change would remove the pain point vs a multi-quarter transformation journey.

References

Airbnb Engineering. (2024). "Chronon, Airbnb's ML Feature Platform, Is Now Open Source." *Airbnb Engineering*. https://medium.com/airbnb-engineering/chronon-airbnbs-ml-feature-platform-is-now-open-source-d9c4dba859e8

Algorithmia. (2020). *2020 State of Enterprise Machine Learning*. Algorithmia. https://www.coriniumintelligence.com/hubfs/0284%20CDAO%20FS/Algorithmia_2020_State_of_Enterprise_ML.pdf

AWS. (2020). "Introducing Amazon SageMaker Feature Store: A Fully Managed Repository to Store, Discover, Share and Serve Machine Learning Features." *AWS What's New*. https://aws.amazon.com/about-aws/whats-new/2020/12/introducing-amazon-sagemaker-feature-store/

Baylor, D., Breck, E., Cheng, H.-T., Fiedel, N., Foo, C. Y., Haque, Z., Haykal, S., Ispir, M., Jain, V., Koc, L., Koo, C. Y., Lew, L., Mewald, C., Modi, A. N., Polyzotis, N., Ramesh, S., Roy, S., Whang, S. E., Wicke, M., Wilkiewicz, J., Zhang, X., and Zinkevich, M. (2017). "TFX: A TensorFlow-Based Production-Scale Machine Learning Platform." *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM. https://doi.org/10.1145/3097983.3098021

Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. (2017). "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction." *2017 IEEE International Conference on Big Data*. IEEE. https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/

Chalk. (n.d.). "Why Your Feature Store Has a Freshness Ceiling." *Chalk Blog*. https://chalk.ai/blog/why-feature-stores-have-freshness-ceiling

Databricks. (2021). "Databricks Launches Feature Store." *Databricks Blog*. https://www.databricks.com/blog/2021/05/27/databricks-announces-the-first-feature-store-integrated-with-delta-lake-and-mlflow.html

Databricks. (2022). "Databricks Feature Store Now Generally Available." *Databricks Blog*. https://www.databricks.com/blog/2022/04/29/announcing-general-availability-of-databricks-feature-store.html

Del Balso, M., and Hermann, J. (2017). "Meet Michelangelo: Uber's Machine Learning Platform." *Uber Engineering*. https://www.uber.com/us/en/blog/michelangelo-machine-learning-platform/

DoorDash. (2020). "Building a Gigascale ML Feature Store with Redis, Binary Serialization, String Hashing, and Compression." *DoorDash Engineering*. https://careersatdoordash.com/blog/building-a-gigascale-ml-feature-store-with-redis/

DoorDash. (2021). "Building Riviera: A Declarative Real-Time Feature Engineering Framework." *DoorDash Engineering*. https://careersatdoordash.com/blog/building-a-declarative-real-time-feature-engineering-framework/

DoorDash. (2022). "Introducing Fabricator: A Declarative Feature Engineering Framework." *DoorDash Engineering*. https://careersatdoordash.com/blog/introducing-fabricator-a-declarative-feature-engineering-framework/

Dowling, J., and Hopsworks. (2024). "A Taxonomy for Data Transformations in AI Systems." *Hopsworks Blog*. https://www.hopsworks.ai/post/a-taxonomy-for-data-transformations-in-ai-systems

Egg, A. (2021). "Online Learning for Recommendations at Grubhub." *Proceedings of the 15th ACM Conference on Recommender Systems*. ACM. https://doi.org/10.1145/3460231.3474599

Etsy. (2024). "Rivulet: Etsy's Realtime Feature Store." *Feature Store Summit 2024*. https://content.hopsworks.ai/hubfs/Feature%20Store%20Summit%202024/FS%20Summit%2024%20-%20Etsy%20-%20Rivulet%20Feature%20Store.pdf

Feast. (2026). "Introduction: What Feast Is and Is Not." *Feast Documentation*. https://docs.feast.dev/

Fennel. (2023). "Read-Write Separation in Fennel's Realtime Feature Store." https://www.linkedin.com/posts/nikhilgarg28_read-write-separation-in-fennels-realtime-activity-7111073468833558528-ApvV

Gallatin, K., and Starr, M. (2023). "The Problem with Timeseries Data in Machine Learning Feature Systems." *Code as Craft, Etsy Engineering*. https://www.etsy.com/codeascraft/the-problem-with-timeseries-data-in-machine-learning-feature-systems

Google Cloud. (2021). "Google Cloud Unveils Vertex AI, One Platform, Every ML Tool You Need." *Google Cloud Blog*. https://cloud.google.com/blog/products/ai-machine-learning/google-cloud-launches-vertex-ai-unified-platform-for-mlops

Hopsworks. (2018). "Introducing the Feature Store: The Data Warehouse for Machine Learning." *Hopsworks News*. https://www.hopsworks.ai/news/introducing-the-feature-store-the-data-warehouse-for-machine-learning

Huyen, C. (2022). *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly Media. https://www.oreilly.com/library/view/designing-machine-learning/9781098107956/

Instacart. (2022). "Lessons Learned: The Journey to Real-Time Machine Learning at Instacart." *Instacart Tech Innovation*. https://www.instacart.com/company/tech-innovation/lessons-learned-the-journey-to-real-time-machine-learning-at-instacart

Kaushal, V. (2022). "Evolution of ML Fact Store." *Netflix Technology Blog*. https://netflixtechblog.com/evolution-of-ml-fact-store-5941d3231762

Khadder, S. (2021). "Feature Stores Explained: The Three Common Architectures." *Redis Blog*. https://redis.io/blog/feature-stores-explained-the-three-common-architectures/

Lathia, N. (2020). "Building an Analytics Feature Store." *Neal Lathia*. https://nlathia.github.io/2020/12/Building-a-feature-store.html

Lim, Y.-X. (2020). "Feast: Bridging ML Models and Data." *Gojek Engineering*. https://medium.com/gojekengineering/feast-bridging-ml-models-and-data-efd06b7d1644

LinkedIn Engineering. (2022). "Open Sourcing Feathr: LinkedIn's Feature Store for Productive Machine Learning." *LinkedIn Engineering Blog*. https://www.linkedin.com/blog/engineering/open-source/open-sourcing-feathr--linkedin-s-feature-store-for-productive-m

Liu, D. (2021). "Twitter's Management of ML Features in Dynamic Environments." *Feature Store Summit 2021*. https://content.hopsworks.ai/hubfs/Feature%20Store%20Summit%20Presentations%20PDFs/Twitter%20-%20Feature%20Store%20Summit.pdf

Liu, Z., Zou, L., Zou, X., Wang, C., Zhang, B., Tang, D., Zhu, B., Zhu, Y., Wu, P., Wang, K., and Cheng, Y. (2022). "Monolith: Real Time Recommendation System With Collisionless Embedding Table." *5th Workshop on Online Recommender Systems and User Modeling, held with the 16th ACM Conference on Recommender Systems*. https://arxiv.org/abs/2209.07663

Mavalankar, A. A., Gupta, A., Gandotra, C., and Misra, R. (2019). "Hotel Recommendation System." *arXiv preprint arXiv:1908.07498*. https://arxiv.org/abs/1908.07498

Monzo. (2022). "Monzo's Machine Learning Stack." *Monzo Blog*. https://monzo.com/blog/2022/04/26/monzos-machine-learning-stack

Nene, A., and Chen, E. (2019). "Michelangelo Palette: A Feature Engineering Platform at Uber." *QCon.ai*. InfoQ. https://www.infoq.com/presentations/michelangelo-palette-uber/

Netflix Technology Blog. (2016). "Distributed Time Travel for Feature Generation." *Netflix Technology Blog*. https://netflixtechblog.com/distributed-time-travel-for-feature-generation-389cccdd3907

Pinterest Engineering. (2025). "A Decade of AI Platform at Pinterest." *Pinterest Engineering*. https://medium.com/pinterest-engineering/a-decade-of-ai-platform-at-pinterest-4e3b37c0f758

Redis. (2021). "Feature Store Summit: The Latest Advancements in Feature Store Technology." *Redis Blog*. https://redis.io/blog/feature-store-summit/

Robinhood. (2021). "How Robinhood Built a Feature Store Using Feast." *apply() Conference*. https://www.tecton.ai/apply/session-video-archive/how-robinhood-built-a-feature-store-using-feast/

Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., and Aroyo, L. M. (2021). "'Everyone Wants to Do the Model Work, Not the Data Work': Data Cascades in High-Stakes AI." *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM. https://doi.org/10.1145/3411764.3445518

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). "Hidden Technical Debt in Machine Learning Systems." *Advances in Neural Information Processing Systems 28*. Curran Associates. https://papers.neurips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems

Sell, T., and Pienaar, W. (2019). "Introducing Feast: An Open Source Feature Store for Machine Learning." *Google Cloud Blog*. https://cloud.google.com/blog/products/ai-machine-learning/introducing-feast-an-open-source-feature-store-for-machine-learning

Shukla, A., Nath, K., Vats, N., Tatipathri, V., and Jagannathan, D. (2023). "D3: An Automated System to Detect Data Drifts." *Uber Engineering*. https://www.uber.com/us/en/blog/d3-an-automated-system-to-detect-data-drifts/

Simha, N. (2020). "Declarative Feature Engineering." *Spark + AI Summit*. https://www.youtube.com/watch?v=LjcKCm0G_OY

Simha, N. (2023). "Chronon: A Declarative Feature Engineering Framework." *Airbnb Engineering*. https://medium.com/airbnb-engineering/chronon-a-declarative-feature-engineering-framework-b7b8ce796e04

Spotify. (2021). "Jukebox." *Feature Store Summit 2021*. https://content.hopsworks.ai/hubfs/Feature%20Store%20Summit%20Presentations%20PDFs/Spotify%20-%20Feature%20Store%20Summit.pdf

Tecton. (2020a). "Founded by Creators of Uber Michelangelo, Tecton.ai Launches to Make World-Class Machine Learning Accessible to Every Company." *GlobeNewswire*. https://www.globenewswire.com/news-release/2020/04/28/2023454/0/en/Founded-by-Creators-of-Uber-Michelangelo-Tecton-ai-Launches-to-Make-World-Class-Machine-Learning-Accessible-to-Every-Company-With-25-Million-From-Andreessen-Horowitz-and-Sequoia.html

Tecton. (2020b). "Tecton Releases Cloud-Native Enterprise Feature Store for Machine Learning." *GlobeNewswire*. https://www.globenewswire.com/news-release/2020/12/07/2140792/0/en/Tecton-Releases-Cloud-Native-Enterprise-Feature-Store-for-Machine-Learning-Raises-35-Million-Series-B-Co-Led-by-Andreessen-Horowitz-and-Sequoia.html

Uber Engineering. (n.d.). "Optimal Feature Discovery for Machine Learning." *Uber Engineering*. https://www.uber.com/us/en/blog/optimal-feature-discovery-ml/

Walecka, S., and Gallatin, K. (2023). "Improving Support for Deep Learning in Etsy's ML Platform." *Code as Craft, Etsy Engineering*. https://www.etsy.com/codeascraft/improving-support-for-deep-learning-in-etsy39s-ml-platform

Yan, E. (2021). "Feature Stores: A Hierarchy of Needs." *ApplyingML.com*. https://applyingml.com/resources/feature-stores/

Zanoyan, V., and Shapiro, E. (2018). "Zipline: Airbnb's Data Management Platform for Machine Learning." *O'Reilly Strata Data Conference*. https://conferences.oreilly.com/strata/strata-ny-2018/cdn.oreillystatic.com/en/assets/1/event/278/Zipline_%20Airbnb_s%20data%20management%20platform%20for%20machine%20learning%20Presentation.pdf

Zinkevich, M. (2017). "Rules of Machine Learning: Best Practices for ML Engineering." *Google Developers*. https://developers.google.com/machine-learning/guides/rules-of-ml